

Implementación de un Proceso Integral de Pruebas de Software bajo Scrum y DevOps: Caso de Estudio en Frappe LMS

Huamani Ollachica, David Alfredo
Universidad Nacional de San
Agustín
Arequipa, Perú
dhuaniano@unsa.edu.pe

Alvarez Choque Miguel Angel
Universidad Nacional de San
Agustín
Arequipa, Perú
malvarezcho@unsa.edu.pe

Huayhua Hillpa, Yourdy Yossimar
Universidad Nacional de San
Agustín
Arequipa, Perú
yhuayhuahi@unsa.edu.pe

Quispe Saavedra, Dennis Javier
Universidad Nacional de San
Agustín
Arequipa, Perú
dquispesaa@unsa.edu.pe

Villafuerte Quispe, Alexander
Universidad Nacional de San
Agustín
Arequipa, Perú
avillafuerteq@unsa.edu.pe

Cuno Salazar Eduardo Joel
Universidad Nacional de San
Agustín
Arequipa, Perú
ecunos@unsa.edu.pe

Quispe Flores, Marco Ramiro
Universidad Nacional de San
Agustín
Arequipa, Perú
mquispeflores@unsa.edu.pe

Resumen —Este artículo presenta el diseño e implementación de un proceso integral y multinivel de aseguramiento de la calidad de software (SQA) bajo el marco ágil Scrum y prácticas DevOps, aplicado sobre la plataforma de código abierto Frappe LMS. La propuesta abarca cuatro niveles de verificación: pruebas unitarias, de integración, de sistema (estrés y seguridad) y de aceptación (E2E), orquestados mediante un pipeline de Integración Continua en GitHub Actions. Los resultados demostraron alta fiabilidad lógica con un 85.71% de cobertura en pruebas unitarias (469 casos exitosos), un 97.7% de efectividad en pruebas funcionales y un 100% de aprobación en pruebas de integración y aceptación de usuario. No obstante, las pruebas de sistema expusieron un cuello de botella por saturación del pool de conexiones de MariaDB a partir de 150 usuarios concurrentes, una tasa de vulnerabilidad del 100% en los 6 vectores de seguridad auditados (IDOR, XSS e inyección de código) y una calificación "E" en fiabilidad por SonarQube. Se concluye que, aunque la automatización CI/CD valida la estabilidad del negocio, se requiere una refactorización urgente en ciberseguridad y optimización de base de datos antes de su despliegue en producción.

Keywords—Software Testing, DevOps, Scrum, Frappe LMS, Continuous Integration (CI/CD), Shift-Left Testing

I. INTRODUCCIÓN

En la actualidad, el Aseguramiento de la Calidad de Software (SQA por sus siglas en inglés), se ha consolidado como un proceso fundamental para el éxito operativo de las organizaciones tecnológicas, considerando que el costo

financiero y reputacional de mitigar fallas en entornos de producción supera cuantiosamente al de su detección en etapas tempranas del ciclo de vida del desarrollo [1].

De forma paralela, el ecosistema de software de código abierto ha experimentado una adopción masiva en diversos sectores, destacando el ámbito educativo, mediante la implementación de Sistemas de Gestión de Aprendizaje (LMS) [5][6]. La confiabilidad, disponibilidad y seguridad de estas plataformas son cruciales, puesto que gestionan flujos masivos de datos de usuarios, evaluaciones académicas y arquitecturas transaccionales [4].

Para equilibrar la rapidez de entrega con la estabilidad del sistema, la industria adopta la sinergia entre la metodología ágil Scrum y las prácticas DevOps [2][3]. Scrum aporta adaptabilidad mediante entregas incrementales en *Sprints*, mientras que DevOps automatiza el ciclo de vida a través de pipelines de Integración y Despliegue Continuos (CI/CD) [1][3]. Esta integración habilita el enfoque *Shift-Left Testing*, transformando las pruebas de software de una fase tardía y manual a una validación automatizada, continua y multinivel [1][7].

Dentro de este contexto, la plataforma Frappe LMS surge como un objeto de estudio idóneo debido a su infraestructura tecnológica basada en el *framework* full-stack Frappe (desarrollado sobre Python y JavaScript) [8].

El presente trabajo de investigación busca diseñar e implementar un proceso integral de pruebas de software automatizadas de cuatro niveles (unitarias, integración,

sistema y aceptación) bajo el marco metodológico Scrum y prácticas DevOps, aplicado sobre la plataforma de código abierto Frappe LMS utilizando el ecosistema de GitHub. El resto del artículo se organiza de la siguiente manera: la Sección II presenta los antecedentes; la Sección III detalla el caso de estudio seleccionado, la Sección IV expone la metodología, la Sección V muestra los resultados; la Sección VI presenta las conclusiones y la Sección VII plantea los trabajos futuros.

II. ANTECEDENTES

A. *Estado del arte*

El estudio del aseguramiento de la calidad del software en entornos ágiles modernos exige una integración sistemática entre los flujos de integración continua (CI/CD) y las estrategias de verificación automatizada. Con el fin de fundamentar esta investigación, se examinan cuatro estudios centrados en el modelado estratégico del testeo continuo.

Ale [12] propuso un marco estratégico para integrar flujos automatizados con pruebas de software mediante una revisión teórica sobre ejecución paralela y entornos virtualizados. Los resultados evidenciaron una detección más temprana de defectos y una reducción en los tiempos de despliegue. Como limitación, destacó la falta de competencias técnicas y recomendó incorporar aprendizaje automático para optimizar el sistema.

Kusumadewi y Adrian [11] realizaron un estudio experimental comparando procesos automatizados y manuales mediante Docker y Jenkins. Los resultados mostraron un rendimiento estable de 93.59 hits/s y un tiempo de respuesta de 47.7 ms bajo carga. Como limitación, el estudio se desarrolló en una red local, por lo que sugirieron evaluar arquitecturas de microservicios distribuidos.

Ghaleb et al. [10] analizaron la evolución de scripts de configuración en 2,557 aplicaciones mediante minería de datos. Encontraron una baja estandarización, donde el 81.3% del esfuerzo se destina a inicializar entornos y solo el 9% a automatizar despliegues. Como limitación, señalaron el alto esfuerzo manual para corregir errores y propusieron asistentes basados en modelos de lenguaje para automatizar estas tareas.

Anwar et al. [13] desarrollaron un marco para integrar la entrega continua con herramientas de análisis de seguridad estático y dinámico. Los resultados incrementaron la detección de vulnerabilidades del 30% al 70% con un impacto temporal del 5%. Entre las limitaciones identificaron los falsos positivos y los problemas de compatibilidad, recomendando incorporar analítica predictiva para mejorar la remediación de riesgos.

La literatura devela una clara progresión desde la validación estratégica e infraestructural [12], [11] hacia la mitigación de la deuda técnica y la consolidación de DevSecOps [10],

[13]. Se constata empíricamente que la eficiencia de la entrega continua depende de la consistencia de sus configuraciones y de la madurez del testware para prevenir regresiones. Esta evidencia justifica científicamente nuestra propuesta orientada a implementar un proceso de pruebas formal (unitarias, integración, sistema y aceptación) bajo Scrum y DevOps en la plataforma open source Frappe LMS. De este modo, se valida el uso estratégico de GitHub Actions como eje orquestador centralizado para automatizar las etapas de verificación y garantizar un producto final robusto antes de su despliegue definitivo.

B. *Tipos de pruebas de software*

1. *Pruebas unitarias*

Su objetivo es verificar, en aislamiento absoluto, el correcto comportamiento de la unidad de código más pequeña de la aplicación. Al ejecutarse directamente sobre el código fuente y simular mediante objetos virtuales cualquier dependencia externa como bases de datos o servicios de red [14].

2. *Pruebas funcionales*

Evalúan que las capacidades del software operen de acuerdo con los requisitos funcionales especificados y los escenarios de uso previstos. Generalmente adoptan un enfoque de caja negra, interactuando con el sistema bajo análisis sin requerir el conocimiento de su estructura lógica interna o de su arquitectura de datos, focalizándose exclusivamente en la correspondencia entre los flujos de entrada y salida [14].

3. *Pruebas de integración*

Analizan la interacción y el flujo de información entre múltiples módulos interdependientes, subsistemas o interfaces de programación de aplicaciones (API) externas. Desempeñan un rol crítico en la validación de la capa de acceso a datos, garantizando que las consultas, las transacciones concurrentes, las colas de mensajería y los mecanismos de almacenamiento en caché operen de manera coordinada sin comprometer la integridad del estado del sistema [14].

4. *Pruebas de sistema*

Validan el comportamiento global del producto de software de forma holística en un entorno que replica con alta fidelidad la configuración física, lógica y de red del ambiente de producción. Su alcance de ejecución de extremo a extremo (end-to-end) garantiza que los componentes de la interfaz de usuario, la lógica de negocio, los sistemas de gestión de bases de datos y las configuraciones de red interactúen correctamente para satisfacer los objetivos sistémicos de la aplicación [14].

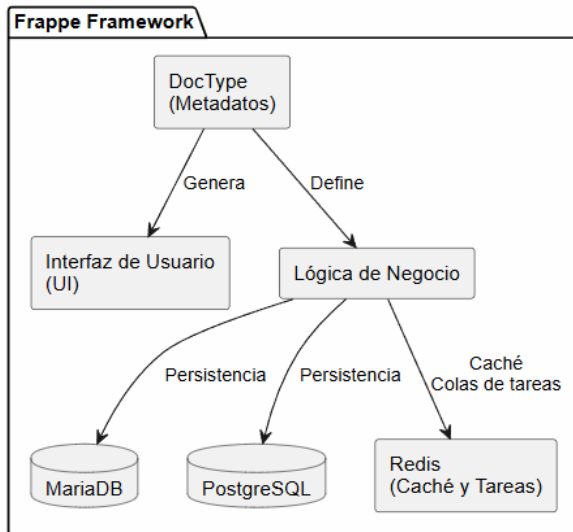
III. CASO DE ESTUDIO

Para este propósito, se seleccionó el ecosistema de software de código abierto desarrollado por *Frappe Technologies* [8]. Esta elección se fundamenta en la arquitectura modular del entorno, su alta escalabilidad y su creciente adopción en el desarrollo de aplicaciones web complejas.

A. Frappe Framework

Frappe Framework es un framework web full-stack de código abierto, desarrollado principalmente en Python y JavaScript. Su arquitectura basada en metadatos utiliza el concepto de DocType, que integra el esquema de la base de datos, la lógica de negocio y la interfaz de usuario en una sola definición. Es compatible con MariaDB y PostgreSQL para la persistencia de datos, y emplea Redis para la caché y la gestión de tareas en segundo plano, simplificando el desarrollo y reduciendo la necesidad de dependencias externas [8]. Frappe LMS. En la Imagen 1 se observa la arquitectura general de Frappe Framework.

Imagen 1: Arquitectura general de Frappe Framework



B. Frappe LMS

Frappe LMS es una plataforma de gestión del aprendizaje de código abierto construida de forma nativa sobre la arquitectura de Frappe Framework. Diseñada bajo un enfoque modular, extiende el uso de los DocTypes para la administración e integración de sus componentes principales, que abarcan la creación y estructuración jerárquica de cursos, capítulos y lecciones multimedia e interactivas. El sistema incorpora un entorno de ejecución de código aislado denominado Falcon para la evaluación automatizada de ejercicios de programación en tiempo real, además de submódulos para el seguimiento del progreso académico, gestión de cohortes (batches), pasarelas de pago independientes y la emisión automatizada de certificaciones personalizadas [9].

C. Componentes identificados para el estudio

Para el proceso de pruebas, la plataforma Frappe Learning (LMS) se descompone en seis componentes funcionales basados en sus metadatos (*DocTypes*) y lógica transaccional.

1. Gestión Académica y de Contenido
2. Evaluación y Cálculo de Progreso
3. Cohortes, Gamificación y Certificación
4. Comercialización y Pagos
5. Infraestructura e Integraciones
6. Comunidad y Analítica

IV. METODOLOGIA

El desarrollo del proyecto fue distribuido en cuatro sprints agrupados en tres hitos. Cada sprint representó una iteración de trabajo con metas específicas y entregables definidos, permitiendo una construcción progresiva y controlada del proceso de aseguramiento de calidad sobre el sistema Frappe LMS.

Cada hito de trabajo tuvo la duración de 15 días, además los roles para cada uno de los integrantes fueron rotativos.

A. Hito 1

La primera fase del proyecto se centró en sentar las bases organizativas y técnicas para el desarrollo del proceso de aseguramiento de la calidad.

Toda la documentación técnica, entregables y artefactos del proyecto fueron centralizados en la Wiki del repositorio principal [31]. Durante esta etapa se definió el tablero Kanban/Scrum, haciendo uso de GitHub Projects, a partir de este artefacto se repartieron los roles y responsabilidades del equipo, conformado por un Test Lead, dos Test Analyst, dos Test Designers y dos Test Architects.

Además se creó una página sencilla en GitHub Pages para la presentación del proyecto, lo cual involucró un análisis del código fuente así como la documentación disponible para la elaboración de dos entregables principales. El Documento de Requisitos de Software [26], el cual recopila las funcionalidades del sistema, además se realizó una división del programa por módulos [15].

Finalmente, se formuló un Plan de Pruebas [16], que estableció los objetivos, niveles, técnicas (principalmente caja negra), herramientas iniciales y asignación de roles para el proyecto.

El primer hito permitió la creación de un ambiente de trabajo colaborativo eficaz, se establece una Estrategia de Ramificación y Pull Request [25],

B. Hito 2

El segundo hito del proyecto estuvo orientado a la ejecución y documentación de las pruebas unitarias. Bajo esta fase se comenzó a implementar las pruebas unitarias definidas en el plan cuyo alcance se limita a la verificación

de la lógica de negocio a nivel de componentes individuales. La ejecución de las pruebas unitarias se encuentran registradas en el Informe de Pruebas Unitarias [22]. La plantilla usada para los resultados se puede observar en la tabla 1.

Tabla 1: Tabla usada para registrar los resultados de las pruebas unitarias.

Campo	Descripción
ID de Prueba	UT-LMS-CRS-001
Objeto de Prueba	
Precondiciones	
Datos de Entrada	
Resultado Esperado	
Resultado Real	
Estado	

Además otro de los objetivos para este sprint fue la creación del Documento de diseño de casos de pruebas funcionales [27]. El cual toma como referencia los requisitos funcionales analizados en el Hito 1. Las pruebas se ejecutaron en un entorno controlado con arquitectura distribuida en Docker (especificada en el Documento de Despliegue [28]), dividida en 3 servicios: Frappe LMS, MariaDB y Redis Alpine. A continuación se mencionan las técnicas aplicadas para este tipo de pruebas.

1. Partición de equivalencia.

Consiste en dividir el dominio de entrada de un componente en clases de equivalencia, donde se espera que el sistema se comporte de manera similar para cualquier valor perteneciente a una misma clase. Luego, se selecciona un único valor representativo por cada clase para construir los casos de prueba.

2. Analisis de valores limite.

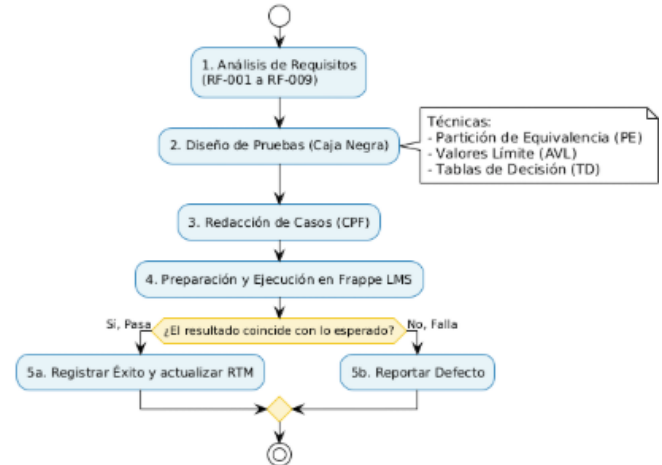
Se enfoca en los valores justo en los límites de cada clase de equivalencia. Generalmente se prueban: el valor límite inferior, el valor inmediatamente inferior, el valor inmediatamente superior al límite, y los equivalentes para el límite superior.

3. Tabla de decisión.

Representa de forma tabular todas las combinaciones posibles de condiciones de entrada y las acciones resultantes esperadas. Es ideal para sistemas con lógica condicional compleja que combina múltiples condiciones booleanas.

El proceso de pruebas funcionales sigue un ciclo de vida clásico de control de calidad, esto se puede observar en la imagen 2.

Imagen 2: Ciclo de vida de las pruebas funcionales

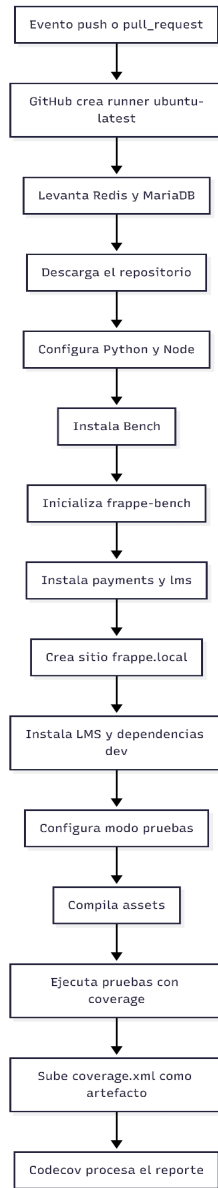


Estas pruebas se registraron en el Informe de Casos de Prueba Funcionales [24], el cual tiene como objetivo documentar la ejecución y dejar constancia de los resultados obtenidos, los defectos identificados y el grado de cumplimiento de los requisitos evaluados.

Por otro lado, se definió el flujo de Despliegue CI/CD Automatizado [29] mediante GitHub Actions, lo cual se documentó exhaustivamente con el propósito de definir la configuración completa para ejecutar de forma automática la suite de pruebas del repositorio. Esta automatización es una pieza clave de la estrategia de Integración Continua (CI), ya que garantiza que cada cambio enviado (a través de eventos de push o pull request dirigidos a las ramas main y develop) sea validado antes de su integración

Basado en la configuración documentada, el flujo de trabajo (workflow) automatizado sigue una secuencia estricta y dependiente, evidenciado en la imagen 3.

Imagen 3: Workflow de Integración Continua (CI) en GitHub Actions.



Por último, se realizó el documento de Plan de Pruebas de Integración [17], el cual define la estrategia, los recursos y los casos de prueba para las pruebas de integración del backend de Frappe LMS. Se eligió un enfoque mixto (Bottom-Up para módulos base y Top-Down para módulos de alto nivel). Y se definió el alcance de las pruebas, la integración de los DocTypes de los 13 módulos funcionales, eventos internos de ciclo de vida (validate, on_update, on_trash), comunicación con instancias de MariaDB y Redis, control de permisos de backend y ejecución dirigida de tareas programadas (schedulers).

La ejecución de estas pruebas, fueron documentadas en el Informe de pruebas de integración [23], cuyo objetivo es consolidar el estado de verificación de las interfaces, eventos, hooks y flujos de datos concurrentes entre los 13 módulos del sistema.

C. Hito 3

El tercer y último hito consistió en la elaboración del Plan de Pruebas de Sistema y Aceptación. Para lo cual se seleccionaron dos tipos de pruebas de orden superior, las cuales son: pruebas de estrés y pruebas de seguridad. Para cada una de las pruebas de sistema se crearon dos planes de pruebas.

El Plan de Pruebas de Estrés [19], consiste en una estrategia de evaluación cuantitativa y cualitativa orientada a determinar los límites de resiliencia, concurrencia y estabilidad de la plataforma Frappe LMS bajo condiciones de carga extrema, se usó Locust, un framework para las pruebas de estrés. las pruebas diseñadas se evidencian en la Tabla 2.

Tabla 2: Pruebas de estrés planificadas.

ID del Escenario	Enfoque Operativo	Recurso / Componente Estresado	Volumen de Carga (Locust)	Métricas Críticas Objetivo
STRESS-ENROLL-001	Inscripciones Concurrentes	Lecturas de BD, consultas agregadas y caché de sesiones en Redis.	200 usuarios / Spawn: 50 u/s	Latencia P99 < 1000 ms; Throughput > 100 req/s.
STRESS-CERT-002	Emisión de Certificados	Escritura concurrente en MariaDB y encolado masivo en la cola RQ short (emails).	200 usuarios / Spawn: 50 u/s	0 certificados duplicados; 0% pérdida de emails.
STRESS-QUIZ-003	Entregas de Quizzes	CPU e inserción transaccional de registros hijos.	200 usuarios / Spawn: 50 u/s	Respetar límite de max_attempts; Latencia P50 < 800 ms.
STRESS-ACCESS-004	Acceso y Progreso de Cursos	Contención de escritura concurrente y lectura masiva.	200 usuarios / Spawn: 50 u/s	Errores de concurrencia (417) controlados e inferiores al 5%.
STRESS-REDIS-005	Saturación de Redis/RQ	Redis como broker de mensajería y persistencia de jobs en colas short y long.	200 usuarios / Spawn: 50 u/s	Latencia de Redis < 10 ms; Uso de memoria < 80% del límite.

El Plan de Pruebas de Seguridad [19] consiste en un marco metodológico dinámico diseñado para auditar la resiliencia del backend de Frappe LMS frente a vectores de ataque dirigidos a la lógica de negocio y al control de acceso. Para la determinación y clasificación de los tipos de riesgo, se adoptaron los estándares de OWASP. La tabla 3 contiene la matriz de pruebas de seguridad definidas.

Tabla 3: Matriz condensada de escenarios de prueba de seguridad.

Identificador	Componente / Endpoint Afectado	Tipo de Riesgo (OWASP)
SEC-LMS-001	get_quiz_with_questions	IDOR / Divulgación de Información
SEC-LMS-002	submit_quiz (Adjuntos)	Stored XSS vía Base64
SEC-LMS-003	extract_package (SCORM)	Inyección de Código Estático / XXE
SEC-LMS-004	validate_image	IDOR / Fuga de Archivos Privados
SEC-LMS-005	create_lms_certificate	Evasión de Lógica de Negocio
SEC-LMS-006	validate_private_attachments	IDOR / Violación de Privacidad

Los resultados de las pruebas de sistema están debidamente documentados en los informes de pruebas de estrés [32] y pruebas de seguridad [33].

Las pruebas de aceptación se estructuraron bajo el Plan de Pruebas de Aceptación [34] y sus resultados se compilaron en el Informe de Pruebas de Aceptación [18]. Estuvieron orientadas a la perspectiva del cuerpo docente, que audita la idoneidad operacional de la plataforma en dos módulos funcionales clave: la gestión de autenticación y sesiones, y la portabilidad y respaldo de contenidos educativos.

Adicionalmente, siguiendo el Plan de Pruebas Automatizadas End-to-End (E2E) [20], se ejecutaron pruebas de extremo a extremo mediante Cypress 12+ integrado en GitHub Actions para validar la interacción entre la interfaz (Vue 3) y el backend (Frappe, Redis, MariaDB). La reproducibilidad se garantiza a través del aislamiento de datos con semillas (*fixtures*) y la simulación transaccional, evitando el uso de pasarelas de pago externas. Los escenarios evaluados y los hallazgos de esta ejecución se consolidan en la Tabla 4 y en el Informe de Resultados de Pruebas Automatizadas End-to-End [21]

Tabla 4: Flujos evaluados en las pruebas automatizadas

Identificador	Flujo Operacional Evaluado
CY-SPEC-01	Creación y estructura de cursos
CY-SPEC-02	Inscripción y tablero de progreso
CY-SPEC-03	Navegación académica (Outline)
CY-SPEC-04	Gestión de cohortes y aforo
CY-SPEC-05	Emisión asíncrona de diplomas

Por último, se realizó un análisis estático con SonarQube y CNES Report integrado en el pipeline de GitHub Actions sobre un volumen de 55,774 líneas de código (70.3% JavaScript, 20.0% Python). Aunque el sistema obtuvo una evaluación aprobatoria general (*Quality Gate Passed*), el diagnóstico detallado (Tabla 5) evidenció brechas críticas en fiabilidad, seguridad y cobertura de pruebas.

Tabla 5: Resumen condensado de métricas de calidad

Dimensión	Estado / Valor	Diagnóstico Técnico
-----------	----------------	---------------------

Mantenibilidad	Rating A	Excelente; ratio de deuda técnica de apenas 0.2%.
Duplicación	0.8%	Óptimo; redundancia mínima (484 líneas).
Fiabilidad	Rating E	Crítico; penalizado por un único bug bloqueante de tipos.
Seguridad	Rating D	Riesgo; 11 vulnerabilidades (3 críticas por ejecución dinámica).
Cobertura	26.6%	Deficiente; 10,820 líneas ejecutables sin verificar.

V. RESULTADOS

El proyecto logró implementar un proceso integral de aseguramiento de calidad para la plataforma Frappe LMS, cumpliendo con los objetivos planteados en cada uno de los tres hitos establecidos, a continuación se van a detallar los resultados por cada tipo de pruebas implementadas.

A. Pruebas unitarias

Se plantearon un total de 469 pruebas únicas. La medición de la cobertura de código se realiza mediante el flujo de Automatización de Pruebas Unitarias con GitHub Actions [30] durante la ejecución continua en el servidor CI/CD. Se obtuvo una tasa de éxito del 100% con una cobertura del 85.71%. En la imagen 4 se puede observar los resultados de la cobertura por Codecov.

Imagen 4: Resultados de cobertura de la rama main.



B. Pruebas funcionales

Se formularon 134 pruebas para los 9 requisitos funcionales analizados, los cuales se pueden observar en la tabla 6.

Tabla 6: Distribución de los casos de prueba funcionales para cada requisito.

Requerimiento	Casos de prueba
RF-001 – Gestión de Cursos	20
RF-002 – Gestión de Enrollments	26
RF-003 – Gestión de Reseñas	20
RF-004 – Gestión de Batches	13
RF-005 – Gestión de Enrollments de Batch	11
RF-006 – Gestión de Feedback de Batch	11
RF-007 – Gestión de Quizzes	19
RF-008 – Gestión de Assignments	15
RF-009 – Gestión de Programming Exercises	19

Los resultados del proceso de validación demostraron un alto índice de conformidad y fiabilidad de la implementación, registrando 131 casos exitosos y únicamente 3 casos fallidos. Porcentaje de éxito: $(131 / 134) \times 100 = 97.7\%$

C. Pruebas de integración

Al haber ejecutado la suite de pruebas bajo aislamiento transaccional utilizando las herramientas nativas de Frappe Bench y aserciones en PyTest/Unittest, se obtuvo un 100% de éxito (Pass Rate) sobre los escenarios diseñados. En la tabla 7 se especifican los resultados.

El porcentaje general de cobertura de código integrado supera el límite de aceptación establecido en el plan de pruebas ($\geq 75\%$), confirmando que el backend está blindado a nivel de servicios.

Tabla 7: Desglose Cuantitativo por Tipo de Prueba de Integración

Categoría	Casos Ejecutados	Casos Aprobados	Casos Fallidos	Tasa de Éxito
Tipo A: API Directa	10	10	0	100%
Tipo B: Eventos y Hooks	8	8	0	100%
Tipo C: Permisos y Seguridad	2	2	0	100%
Tipo D: Manejo de Errores	3	3	0	100%
Tipo E: Caché e Índices	4	4	0	100%
TOTAL GLOBAL	27	27	0	100%

D. Pruebas de estrés

La ejecución de las pruebas de estrés se realizó mediante Locust 2.45.0 sobre un entorno de integración continua en GitHub Actions, replicando la arquitectura de producción con tres instancias de Redis (caché, colas RQ y SocketIO) y MariaDB 10.8 como motor de base de datos. Se diseñaron cinco escenarios complementarios (Tabla 2), cada uno con datos de prueba específicos sembrados antes de la ejecución. Todos los escenarios se sometieron a una carga de 200 usuarios concurrentes con una tasa de incorporación de 50 usuarios por segundo durante 120 segundos. La Tabla 8 sintetiza los resultados agregados de los cinco escenarios.

Tabla 8: Resultados agregados de los cinco escenarios de estrés.

Escenario	Endpoint principal	Requests	Fallos reales	P50	Throughput
STRESS-ENROLL-001	Lecturas agregadas (GETs)	27,126	0%	100 ms	226 req/s
STRESS-CERT-T-002	'create_certificate' (POST)	19,643	0%	340 ms	163 req/s
STRESS-QUIZ-003	'submit_quiz' (POST)	4,180	0.03%	3700 ms	34 req/s
STRESS-ACC-ESS-004	'save_progress' + GETs	27,307	0%	100 ms	227 req/s
STRESS-REDIS-005	'create_certificate' + 'save_progress'	29,339	0%	75 ms	244 req/s

Se registraron 49 fallos de login en cada escenario corresponden en su totalidad a la saturación del pool de conexiones de MariaDB, evidenciada por el error `'MySQLdb.OperationalError: (1040, 'Too many connections')'` al superar los 150 usuarios concurrentes. Las Tablas 8 y 9 presentan el desglose detallado del escenario STRESS-ENROLL-001 por endpoint y las fases de degradación observadas.

Tabla 9: Fases de degradación del sistema bajo carga creciente.

Fase	Usuarios Concurrentes	Comportamiento Observado
Normal	0 – 50	Login < 1 s; endpoints < 100 ms (P50).
Degradación	50 – 150	Login escala a 7 – 10 s; endpoints se mantienen entre 200 – 400 ms (P50)
Saturación	150 – 200	Login falla en un 24.5% por agotamiento del pool de conexiones MariaDB; endpoints GET/POST permanecen funcionales pero con P99 elevado.

El análisis de los cinco escenarios permitió identificar tres resultados importantes. Primero, el endpoint `create_certificate` presentó problemas cuando muchos usuarios intentaron generar un certificado al mismo tiempo, ya que la validación y el guardado no se realizan como una sola

operación. Esto produjo numerosos errores (HTTP 417) y, en un entorno con varios servidores, podría ocasionar certificados duplicados. Segundo, `submit_quiz` fue el endpoint con peor rendimiento, ya que el procesamiento de las respuestas consume más tiempo y recursos que el resto de las operaciones, reduciendo la cantidad de solicitudes que puede atender por segundo. Finalmente, se comprobó que los mecanismos de protección del sistema, como el límite de solicitudes y el control de concurrencia, funcionaron correctamente bajo carga, evitando la corrupción de datos. Además, Redis mantuvo un rendimiento estable durante todas las pruebas de estrés.

E. Pruebas de seguridad

La ejecución de las pruebas de seguridad evidenciaron una tasa de vulnerabilidad del 100% en los 6 escenarios evaluados. Este resultado confirma que todas las brechas críticas y altas del modelo de amenazas son completamente explotables debido a la falta de sanitización de entradas y fallos en el control de acceso a objetos (IDOR).

Tabla 10: Métricas cuantitativas de la auditoría de seguridad en el backend.

Métrica de Seguridad Evaluada	Valor Obtenido	Significado e Impacto Técnico
Total de Vectores Auditados	6	Cobertura desde SEC-LMS-001 hasta SEC-LMS-006.
Controles Nativos Exitosos	0	Ningún filtro bloqueó las cargas maliciosas.
Pruebas con Confirmación de Brecha	6	Explotación exitosa de las debilidades del backend.
Tasa de Vulnerabilidad General	100%	Exposición total ante vectores de ataque remotos.
Severidad Máxima Registrada	Crítica	Compromiso de datos privados e inyección de código.

F. Pruebas de aceptación

Se ejecutó el 100% de los escenarios planificados (4 de 4 casos), obteniendo una tasa de éxito del 100% sin registro de incidencias técnicas o funcionales en el gestor de defectos.

La síntesis cuantitativa y cualitativa de las pruebas de aceptación ejecutadas se consolida en la Tabla 11.

Tabla 11: Matriz de resultados de las pruebas de aceptación de usuario (UAT) para el rol docente.

Identificador	Módulo Funcional	Escenario Operacional Evaluado	Comportamiento Registrado y Tiempo de Respuesta	Estado de Conformidad
UAT-01	Autenticación	Registro e inicio de sesión exitoso	Autenticación de credenciales nominales, resolución de sesión y redirección sincrónica al panel principal.	Aprobado
UAT-02	Autenticación	Intento de inicio de sesión fallido	Denegación de acceso ante credenciales inválidas y despliegue de alerta ("Invalid credentials, try again.").	Aprobado
UAT-03	Portabilidad	Exportación e importación de estructura relacional en curso completo	Generación y deserialización de paquete .zip en un tiempo de ejecución de 45 s.	Aprobado
UAT-04	Portabilidad	Carga de paquete corrupto o incompatible	Interrupción de transacción o (fast-fail) en 3 s. abortando la carga ("Please upload a valid CSV file").	Aprobado

VI. CONCLUSIONES

La implementación de un proceso integral de pruebas bajo el marco metodológico Scrum y las prácticas DevOps demostró ser una estrategia altamente efectiva para orquestar el aseguramiento de la calidad en la plataforma de código abierto Frappe LMS. La integración continua mediante GitHub Actions permitió materializar con éxito el enfoque Shift-Left Testing, logrando automatizar la validación multinivel a lo largo del ciclo de vida del desarrollo.

A nivel de verificación lógica y funcional, los resultados evidenciaron una notable madurez en los componentes centrales del sistema. Las pruebas unitarias alcanzaron una cobertura del 85.71% con una tasa de éxito del 100% en los 469 escenarios evaluados, mientras que las pruebas funcionales registraron un índice de aprobación del 97.7%. De manera complementaria, las pruebas de integración confirmaron la correcta interacción de los módulos, bases de datos (MariaDB) y caché (Redis), obteniendo un 100% de éxito y garantizando que el *backend* se encuentre consolidado a nivel de servicios.

No obstante, las pruebas de sistema y el análisis estático de código revelaron deficiencias arquitectónicas críticas en materia de ciberseguridad y fiabilidad. La auditoría de seguridad demostró una tasa de vulnerabilidad del 100% en los seis escenarios evaluados, exponiendo fallos severos asociados a la ausencia de sanitización de entradas y controles de acceso a objetos (IDOR). Asimismo, la evaluación con SonarQube penalizó al sistema con una calificación "E" en fiabilidad debido a un *bug* bloqueante, y constató una deficiente cobertura global de código ejecutable del 26.6%. En síntesis, si bien el *pipeline* automatizado validó la viabilidad operativa y la resiliencia de la lógica de negocio, el producto requiere una refactorización urgente orientada a mitigar las

vulnerabilidades expuestas antes de su despliegue en entornos de producción.

VII. TRABAJOS FUTUROS

A partir de los hallazgos expuestos, se plantean las siguientes líneas de acción para futuras investigaciones y desarrollos.

Se requiere con urgencia la mitigación sistemática de las brechas críticas de seguridad identificadas (IDOR, *Stored XSS* e Inyecciones de Código). Para ello, se propone evolucionar el *pipeline* actual hacia una arquitectura DevSecOps, integrando herramientas avanzadas de Análisis Dinámico (DAST) que validen de forma continua la sanitización de *endpoints* y las políticas de acceso.

VIII. REFERENCIAS

- [1] Y. Jani, "Implementing Continuous Integration and Continuous Deployment (CI/CD) in Modern Software Development," *International Journal of Science and Research (IJSR)*, vol. 12, no. 6, pp. 2984–2987, Jun. 2023, doi: 10.21275/sr24716120535.
- [2] T. I. Azonuche, M. E. Aigbogun, and J. O. Enyejo, "Investigating Hybrid Agile Frameworks Integrating Scrum and Devops for Continuous Delivery in Regulated Software Environments," *Int J Innov Sci Res Technol*, pp. 810–824, Apr. 2025, doi: 10.38124/ijisrt/25apr1164.
- [3] I. C. Donca, O. P. Stan, M. Misaros, D. Gota, and L. Miclea, "Method for Continuous Integration and Deployment Using a Pipeline Generator for Agile Software Projects," *Sensors*, vol. 22, no. 12, Jun. 2022, doi: 10.3390/s22124637.
- [4] A. Ghosh, A. Nafalski, Z. Nedic, and A. P. Wibawa, "Learning management systems with emphasis on the Moodle at UniSA," *Bulletin of Social Informatics Theory and Application*, vol. 3, no. 1, pp. 13–21, May 2019, doi: 10.31763/businta.v3i1.160.
- [5] N. Pankaja and M. Raj, "Volume-02, Issue-07," *American Journal of Engineering Research (AJER)*, pp. 124–130, 2013, [Online]. Available: www.ajer.org
- [6] E. Yılmaz, A. Elen, E. C. Kuran, E. Kaya, and E. Dönmez, "A Comparison of Open-Source Learning Management Systems Used for Distance Education in Higher Education Institutions," *International Scientific and Vocational Studies Journal*, vol. 8, no. 2, pp. 211–222, Dec. 2024, doi: 10.47897/bilmes.1597768.
- [7] A. Venkitaraman, "Early Bug Detection through Shift Left Testing," *International Journal of Innovative Science and Research Technology (IJSRT)*, pp. 185–190, Nov. 2024, doi: 10.38124/ijisrt/ijisrt24nov177.
- [8] Frappe Technologies, "Frappe Framework Source Code and Technical Documentation," GitHub Repository, 2026. [Online]. Available: <https://github.com/frappe/frappe>.
- [9] Frappe Technologies, "Frappe LMS Source Code and Documentation," *GitHub Repository*, 2026. [Online]. Available: <https://github.com/frappe/lms>.
- [10] T. A. Ghaleb, O. Abduljalil, and S. Hassan, "CI/CD Configuration Practices in Open-Source Android Apps: An Empirical Study," May 2025, [Online]. Available: <http://arxiv.org/abs/2411.06077>
- [11] R. Kusumadewi and R. Adrian, "Performance Analysis of Devops Practice Implementation Of CI/CD Using Jenkins," *MATICS: Jurnal Ilmu Komputer dan Teknologi Informasi (Journal of Computer Science and Information Technology)*, vol. 15, no. 2, pp. 90–95, Oct. 2023, doi: 10.18860/mat.v15i2.17091.
- [12] N. Kumar Ale, "Integrating Continuous Integration / Continuous Deployment (CI/CD) with Test Automation: Enhancing Software Development Efficiency," *International Journal of Science and Research (IJSR)*, vol. 11, no. 1, pp. 1636–1638, Jan. 2022, doi: 10.21275/sr24627045933.
- [13] S. Anwar and C. Author, "Software Engineering in Computing Systems Framework for Integrating Continuous Integration and Continuous Deployment (CI or CD) with Automated Security Testing to Improve Software Dependability," 2025, [Online]. Available: <https://journal.apjikom.or.id/index.php/SECONS>
- [14] G. J. Myers, T. Badgett, and C. Sandler, *The Art of Software Testing*, 3rd ed. Hoboken, NJ, USA: John Wiley & Sons, 2012.
- [15] "Módulos del proyecto," GitHub, <https://github.com/PS-Grupo-7-Gorgojo/lms/wiki/M%C3%B3dulos-del-proyecto>
- [16] "Plan de Pruebas Unitarias," GitHub, <https://github.com/PS-Grupo-7-Gorgojo/lms/wiki/Plan-de-Pruebas-Unitarias>
- [17] "Plan de Pruebas de Integración," GitHub, <https://github.com/PS-Grupo-7-Gorgojo/lms/wiki/Plan-de-Pruebas-de-Integraci%C3%B3n>
- [18] "Informe de Pruebas de Aceptación," *GitHub Wiki*. [En línea]. Disponible en: <https://github.com/PS-Grupo-7-Gorgojo/lms/wiki/Informe-de-Pruebas-de-Aceptaci%C3%B3n>
- [19] "Plan de Pruebas de Sistema," GitHub, <https://github.com/PS-Grupo-7-Gorgojo/lms/wiki/Plan-de-Pruebas-de-Sistema>
- [20] "Plan de Pruebas Automatizadas End-to-End (E2E)," GitHub, [https://github.com/PS-Grupo-7-Gorgojo/lms/wiki/Plan-de-Pruebas-Automatizadas-End-to-End%E2%80%90%E2%80%90End-\(E2E\)](https://github.com/PS-Grupo-7-Gorgojo/lms/wiki/Plan-de-Pruebas-Automatizadas-End-to-End%E2%80%90%E2%80%90End-(E2E))
- [21] "Informe de Resultados de Pruebas Automatizadas End-to-End (E2E)," GitHub, [https://github.com/PS-Grupo-7-Gorgojo/lms/wiki/Informe-de-Resultados-de-Pruebas-Automatizadas-End-to-End%E2%80%90%E2%80%90End-\(E2E\)](https://github.com/PS-Grupo-7-Gorgojo/lms/wiki/Informe-de-Resultados-de-Pruebas-Automatizadas-End-to-End%E2%80%90%E2%80%90End-(E2E))
- [22] "Informe de pruebas unitarias," GitHub, <https://github.com/PS-Grupo-7-Gorgojo/lms/wiki/Informe-de-pruebas-unitarias>

- [23] “Informe de pruebas de integración,” GitHub,
<https://github.com/PS-Grupo-7-Gorgojo/lms/wiki/Informe-de-pruebas-de-integraci%C3%B3n>
- [24] “Informe de Casos de Prueba Funcionales,” GitHub,
<https://github.com/PS-Grupo-7-Gorgojo/lms/wiki/Informe-de-Casos-de-Prueba-Funcionales>
- [25] “Estrategia de Ramificación y Pull Requests,” GitHub,
<https://github.com/PS-Grupo-7-Gorgojo/lms/wiki/Estrategia-de-Ramificaci%C3%B3n-y-Pull-Requests>
- [26] “Documento Requisitos de Software,” GitHub,
<https://github.com/PS-Grupo-7-Gorgojo/lms/wiki/Documento-Requisitos-de-Software>
- [27] “Documento de diseño de casos de pruebas,” GitHub,
<https://github.com/PS-Grupo-7-Gorgojo/lms/wiki/Documento-de-dise%C3%B1o-de-casos-de-pruebas>
- [28] “Documento de Despliegue,” GitHub,
<https://github.com/PS-Grupo-7-Gorgojo/lms/wiki/Documento-de-Despliegue>
- [29] “Despliegue CI CD Automatizado,” GitHub,
<https://github.com/PS-Grupo-7-Gorgojo/lms/wiki/Despliegue-CI-CD-Automatizado>
- [30] “Automatización de Pruebas Unitarias con GitHub Actions,” *GitHub Wiki*. [En línea]. Disponible en:
<https://github.com/PS-Grupo-7-Gorgojo/lms/wiki/Automatizaci%C3%B3n-de-Pruebas-Unitarias-con-GitHub-Actions>
- [31] “Home,” *GitHub Wiki*. [En línea]. Disponible en:
<https://github.com/PS-Grupo-7-Gorgojo/lms/wiki>
- [32] “Informe de Pruebas de Estrés,” *GitHub Wiki*. [En línea]. Disponible en:
<https://github.com/PS-Grupo-7-Gorgojo/lms/wiki/Informe-de-Pruebas-de-estr%C3%A9s>
- [33] “Informe de Pruebas de Seguridad,” *GitHub Wiki*. [En línea]. Disponible en:
<https://github.com/PS-Grupo-7-Gorgojo/lms/wiki/Informe-de-Pruebas-de-Seguridad>
- [34] “Plan de Pruebas de Aceptación,” *GitHub Wiki*. [En línea]. Disponible en:
<https://github.com/PS-Grupo-7-Gorgojo/lms/wiki/Plan-de-Pruebas-de-Aceptaci%C3%B3n>